

Contents

Cursor IDE: Comprehensive Architecture Document	3
Table of Contents	3
1. Overview	3
What Is Cursor	3
The Company: Anysphere	3
Funding and Growth	4
Key Metrics (Late 2025)	4
Timeline	4
2. Architecture	5
The VS Code Fork Decision	5
High-Level Data Flow	5
3. Core Features	6
3.1 Tab (AI Autocomplete)	6
3.2 Chat	6
3.3 Composer (Multi-File Agent Editing)	7
3.4 Agent Mode	7
4. Context Engine	7
How Codebase Indexing Works	7
5. Model Support	8
Built-in Models	8
Custom API Keys	8
Auto Mode	8
6. Rules System	9
Overview	9
Rule Types	9
File Locations	9
MDC Format	9
Generating Rules	9
7. MCP Support	10
Overview	10
Supported Transport Types	10
Configuration	10
Capabilities and Limitations	10
Available Integrations	10
Comparison with Claude Code MCP	10
8. Agent Mode Deep Dive	10
Architecture	10
Tool Access	11
YOLO Mode	11
Checkpoints	11
Multi-Agent Parallel Execution (Cursor 2.0+)	11
Comparison with Claude Code Agent	11
9. Extensions & Marketplace	11
VS Code Extension Compatibility	11
Cursor-Specific Features	12
Migration from VS Code	12
10. Terminal Integration	12
AI-Powered Terminal	12
Sandboxed Terminals	12
Terminal in Agent Mode	12
11. Multi-File Editing	12
How Composer Handles Multi-File Edits	12
Diff Rendering	13
Fast Apply	13
Atomic Operations	13

12. Session Management	13
Current State	13
Known Issues	13
Workarounds	13
Comparison with Claude Code	13
13. Collaboration & Team Features	14
Team Rules	14
Custom Commands	14
Admin Controls	14
Enterprise Dashboard	14
14. Background Agents	14
Overview	14
How They Work	14
Environment	14
Configuration Requirements	15
Parallel Agents (Cursor 2.0+)	15
Linear Integration (Cursor 1.5)	15
Comparison with Claude Code Background Agents	15
15. Bugbot (PR Review)	15
Overview	15
How It Works	15
Configuration	15
Effectiveness	16
16. Cursor CLI	16
Overview	16
Commands	16
Capabilities	16
Comparison with Claude Code CLI	16
17. Pricing	16
Tier Comparison (as of early 2026)	16
Credit System (Post-June 2025)	17
Approximate Requests per \$20 Credit Pool	17
Auto Mode Pricing	17
18. Strengths	17
1. Visual, Low-Friction Editing	17
2. Multi-File Composer	17
3. VS Code Familiarity	18
4. Background Agents	18
5. Tab Autocomplete Quality	18
6. Codebase Context	18
7. Built-in Browser Tool	18
8. Bugbot	18
9. Enterprise Features	18
10. Rapid Innovation	18
19. Limitations	18
1. Effective Context Window	18
2. Extension Ecosystem Friction	18
3. No Persistent Memory	18
4. Performance on Large Codebases	18
5. Privacy Concerns	19
6. Closed Source	19
7. Upstream Merge Burden	19
8. AI Reliability	19
9. UI Clutter	19
10. No SDK/Programmatic API	19
11. Enterprise Maturity Gaps	19
20. Version History	19

Key Releases Timeline	19
Milestone Timeline	20
21. Cursor vs Claude Code	20
Architectural Philosophy	20
Head-to-Head Comparison	20
Where Each Excels	21
The Emerging Consensus	21
Sources	21

Cursor IDE: Comprehensive Architecture Document

Last updated: March 25, 2026 **Subject:** Cursor by Anysphere – AI-native code editor, architecture, features, and competitive analysis

Table of Contents

1. Overview	
2. Architecture	
3. Core Features	
4. Context Engine	
5. Model Support	
6. Rules System	
7. MCP Support	
8. Agent Mode Deep Dive	
9. Extensions & Marketplace	
10. Terminal Integration	
11. Multi-File Editing	
12. Session Management	
13. Collaboration & Team Features	
14. Background Agents	
15. Bugbot (PR Review)	
16. Cursor CLI	
17. Pricing	
18. Strengths	
19. Limitations	
20. Version History	
21. Cursor vs Claude Code	

1. Overview

What Is Cursor

Cursor is an AI-native code editor built by **Anysphere**, a San Francisco-based startup. Rather than building an IDE from scratch, Anysphere forked Visual Studio Code and embedded AI deeply into every layer of the editing experience – autocomplete, chat, multi-file editing, terminal, and autonomous agent workflows. The result is a product that looks and feels like VS Code but behaves fundamentally differently, with AI as a first-class citizen in every interaction.

The Company: Anysphere

Anysphere was founded in **2022** by four MIT classmates, all under 30 years old:

Founder	Role	Background
Michael Truell	CEO	USA Computing Olympiad finalist; interned at Google, Two Sigma, Octant; built a programming game at age 14
Sualeh Asif	CPO	From Pakistan; International Mathematical Olympiad representative; IBM intern (neural machine translation)
Arvid Lunnemark	Co-founder	International Olympiad in Informatics medalist; roles at Stripe, Jane Street, QuantCo
Aman Sanger	Co-founder	MIT CS; early AI research; key architect of Cursor’s inference stack

The four met while studying computer science and mathematics at MIT, collaborated on research projects, and participated in the Neo Scholars mentorship program for technical undergraduates.

Funding and Growth

Date	Round	Amount	Valuation	Lead Investors
April 2022	Pre-seed	\$400K	–	Angels
October 2023	Seed	\$8M	–	OpenAI Startup Fund, Nat Friedman, Arash Ferdowsi
~2024	Series A/B	\$100M	\$2.6B	–
June 2025	Series C	\$900M	\$9.9B	Thrive Capital
November 2025	Series D	\$2.3B	\$29.3B	Accel, Coatue Management (with Google, Nvidia)

Total funding: ~\$3.5 billion as of late 2025.

Key Metrics (Late 2025)

- **1M+ daily active users**
- **50K+ businesses** using Cursor
- Over **50% of Fortune 500** companies had adopted Cursor by mid-2025
- Surpassed **\$1 billion ARR** by late 2025
- **9,900% year-over-year** ARR growth at one point
- **300+ employees**
- The Series D made all four founders billionaires

Timeline

- **2022:** Founded, pre-seed funding
- **March 2023:** Cursor launched publicly (graduated from OpenAI accelerator)
- **January 2025:** Crossed \$100M ARR (reportedly with zero marketing spend)
- **October 2025:** Cursor 2.0 released with Composer model and multi-agent architecture
- **November 2025:** \$29.3B valuation at Series D
- **2026:** Continued rapid iteration, Cursor 2.3+, Composer 2, visual designer features

2. Architecture

The VS Code Fork Decision

Cursor is a **full fork** of Visual Studio Code – not a plugin or extension. This architectural decision is the foundation of everything that makes Cursor different from tools like GitHub Copilot (which runs as a VS Code extension).

By forking VS Code, Anysphere gained access to the C++ and TypeScript internals of the editor, giving them what amounts to “root access” to the editor engine. This enables capabilities that are physically impossible for any VS Code extension:

Shadow Workspace Cursor can spawn **hidden, parallel instances** of the editor engine to validate code changes in the background. When the AI writes code, Cursor can open a shadow workspace, apply the edits there, run the Language Server Protocol (LSP) to check for lints and type errors, and report results back – all without touching your visible workspace. This lets the AI iterate on its own output before presenting it to you.

Native Diff Rendering Instead of clunky side-by-side diff windows, Cursor renders AI suggestions as **inline color-coded overlays** directly in the active file. The user sees green (additions) and red (deletions) in-place, and can accept or reject with a single keystroke.

Terminal Interception Cursor can natively read terminal output and inject commands. When a compilation fails, the AI can see the exact error message from the terminal, generate a fix, apply it, and re-run – all automatically in “YOLO mode.”

Fast Apply Model Cursor developed a specialized model for rapidly applying code changes:

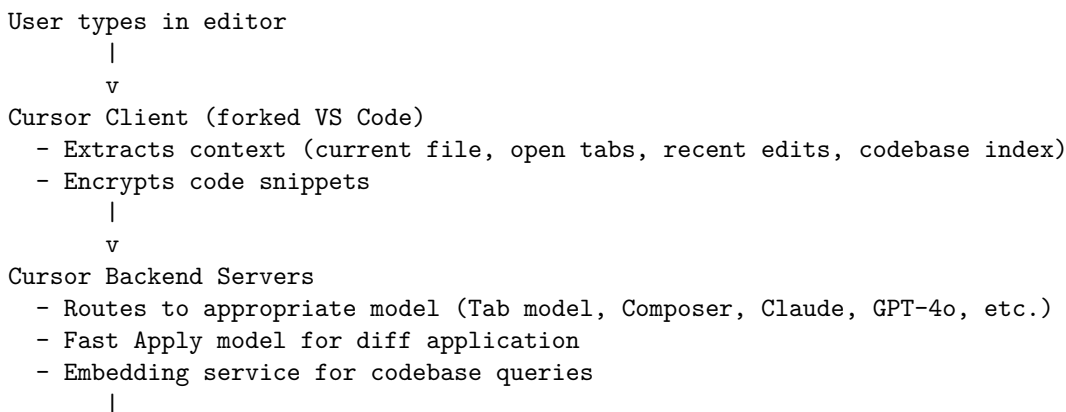
- Built on a **70-billion-parameter Llama base** model
- Fine-tuned on Cursor-specific data (Cmd+K edit instructions and corresponding diffs)
- Runs on **Fireworks AI** inference infrastructure
- Achieves **~1,000 tokens/second** using a technique called **speculative edits**
- Outperforms GPT-4 and GPT-4o at the specific task of applying code diffs

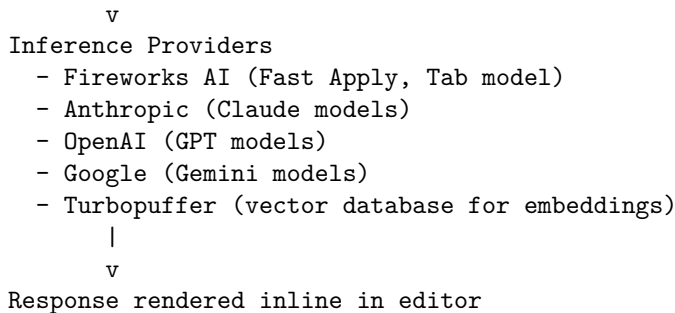
Speculative edits is Cursor’s variant of speculative decoding. Instead of using a smaller draft model, a deterministic algorithm speculates on future tokens based on the existing code (since most of a file remains unchanged during an edit). This allows dramatically longer speculation runs than standard speculative decoding.

Processing Architecture

- The backend sees **1M+ queries per second**, primarily from tiny autocomplete (Tab) requests
- Code snippets are encrypted before transmission to Cursor’s servers
- The Fast Apply model handles full-file rewrites for files under ~400 lines, and diff-based approaches for larger files
- A dedicated team exists solely for **merging upstream VS Code updates** (VS Code releases monthly)

High-Level Data Flow





3. Core Features

3.1 Tab (AI Autocomplete)

What it is: Cursor Tab is a proprietary AI autocomplete system that goes far beyond traditional code completion.

How it works: 1. As you type, Cursor extracts a small window of relevant code context 2. Context is encrypted and sent to Cursor's backend 3. The proprietary **Tab model** predicts your next action 4. Suggestions are returned and displayed inline in the editor

What makes it different from Copilot: - **Edit prediction, not just insertion:** Tab can predict changes to existing lines, not only append new text. It treats coding as a series of intent-driven edits. - **Multi-line awareness:** When you pause, Tab predicts not just the next token but potentially the next several lines or the next edit you intend to make. - **Cursor position prediction:** Tab can anticipate where your cursor should move next after an edit. - **Cross-file session awareness:** Completions account for code you have written in other files during the current session. - **Learning from acceptance/rejection:** The more you accept (Tab) or reject (Esc), the better suggestions become within a session.

The Fusion Tab model (introduced in v0.45) improved codebase understanding and context-aware prediction quality.

3.2 Chat

What it is: An AI chat interface integrated into the editor sidebar, with rich context injection via @ symbols.

Context symbols available:

Symbol	Purpose
@File	Reference a specific file (e.g., @package.json)
@Folder	Reference an entire directory
@Codebase	Semantic search across the entire indexed project
@Code	Reference specific code snippets (more granular than @File)
@Docs	Reference official documentation for libraries/frameworks; add custom docs by URL
@Web	Perform a live web search and inject results into context
@Git	Reference git commits, diffs, or pull requests
@Commit	Current working state changes vs last commit
@Branch	Compare current branch changes vs main

How Chat differs from Composer/Agent: Chat was originally a separate mode for Q&A and exploration. With Cursor 2.0, Chat and Composer were merged into a single **Agent** interface, though the underlying modes still differ in capability.

3.3 Composer (Multi-File Agent Editing)

What it is: Composer is Cursor's multi-file editing engine and, since 2.0, also the name of their proprietary coding model.

Composer the Feature: - Allows instructing AI to make coordinated edits across multiple files simultaneously - Can create new files, delete files, and preview all changes before applying - Renders diffs inline with accept/reject per file or per hunk - Works with the full codebase context to understand dependencies between files

Composer the Model (released with Cursor 2.0, October 2025): - A **mixture-of-experts** architecture trained through reinforcement learning - Trained inside real codebases where it learned to use actual development tools (semantic search, file editors, terminal commands) - Completes most interactive turns in **under 30 seconds** - ~4x faster generation speed compared to similarly capable frontier models - Optimized for throughput and responsiveness over raw token-perplexity benchmarks - **Composer 2** released March 2026 with further performance improvements

3.4 Agent Mode

What it is: Agent is the default and most autonomous mode in Cursor. It plans multi-step tasks, edits multiple files, runs terminal commands, and iterates until tests pass or errors resolve.

How it works: 1. **Analysis:** Analyzes your request and the codebase context to comprehend the task 2. **Discovery:** Searches through codebase, documentation, and the web to identify relevant files 3. **Planning:** Breaks the task into smaller steps and plans changes 4. **Execution:** Makes code modifications across the entire codebase 5. **Validation:** Runs linters, tests, and terminal commands to verify correctness 6. **Iteration:** If errors are found, iterates on fixes automatically

Tools available to Agent: - File operations (read, write, create, delete, find by fuzzy name matching) - Terminal command execution (with sandboxing) - Web search - Linting and error detection - Codebase semantic search - Browser tool (for testing web applications)

Safety: Agent creates checkpoints before making changes, allowing rollback if needed.

4. Context Engine

How Codebase Indexing Works

Cursor's context engine is built on a **semantic search pipeline** that indexes your entire codebase and retrieves relevant code at query time.

Step 1: File Monitoring via Merkle Trees Cursor uses a **Merkle tree** data structure to efficiently track file changes: - Every file is represented as a leaf node with a cryptographic hash - Non-leaf nodes contain hashes of their children - Every **10 minutes**, Cursor checks for hash mismatches - Only changed files are re-processed, dramatically reducing bandwidth

Step 2: Chunking When a file changes, Cursor splits it into **syntactic chunks** – meaningful code segments based on the language's syntax (functions, classes, blocks, etc.) rather than arbitrary line counts.

Step 3: Embedding Each chunk is converted into a vector embedding using an embedding model (likely OpenAI's embedding models or custom code-optimized models). Cursor **caches embeddings by chunk content**, so unchanged chunks hit the cache and skip re-computation.

Step 4: Storage in Turbopuffer Embeddings are stored in **Turbopuffer**, Cursor's vector database: - Each vector is stored with metadata: start/end line numbers, obfuscated file path - File paths are **obfuscated** using a client-stored secret key and deterministic 6-byte nonce - Path segments (split by / and .) are encrypted individually - This enables path-based filtering without exposing plaintext paths on the server

Step 5: Retrieval at Query Time When you use @Codebase or the AI needs codebase context: 1. Cursor computes an embedding for your query 2. The embedding is sent to Turbopuffer for nearest-neighbor search 3. Results return with obfuscated file paths and line ranges 4. The Cursor client reads the actual code content from your **local files** (code never stored on server for privacy mode users) 5. Retrieved chunks are sent as context to the LLM

Privacy Model

- For **Privacy Mode** users, no plaintext code is stored on Cursor’s servers or in Turbopuffer
- Only obfuscated paths and embeddings are stored remotely
- Actual code retrieval happens locally on your machine
- The `.cursorignore` file controls which files are excluded from indexing

Context Assembly Priority Cursor assembles context from multiple sources with intelligent prioritization: 1. **Workspace index** – semantic index of the entire codebase 2. `.cursor/rules/ files` – project-specific instructions 3. **@ mentions** – explicit context injected by the user 4. **Open tabs and recent edits** – implicit context from current work 5. **LSP data** – type information, definitions, references from language servers

5. Model Support

Built-in Models

Cursor ships with a curated list of models accessible through its proxy:

Provider	Models	Notes
Anthropic	Claude Sonnet 4.5, Claude Opus, Claude Haiku	Claude Sonnet 4.5 recommended for best quality/cost balance
OpenAI	GPT-4o, GPT-4o-mini, o4-mini, o3	o1/o1-mini/o3-mini not supported with custom keys
Google	Gemini 2.0 Flash, Gemini 1.5 Flash 500k	Budget-friendly; ~550 requests per \$20 vs ~225 for Claude
Cursor	Composer model, Tab model, Fast Apply model	Proprietary models for specific tasks
Auto	Cursor’s automatic model selection	Unlimited on paid plans; routes to cost-optimized models

Custom API Keys

Users can bring their own API keys from: - **OpenAI** (direct API) - **Anthropic** (direct API) - **Google** (Gemini API) - **Azure OpenAI** - **AWS Bedrock** - **Any OpenAI-compatible endpoint** (for self-hosted or third-party models like DeepSeek)

When using custom keys: - Keys are stored **locally on your machine** and never uploaded to Cursor’s servers - Requests go **directly to the provider**, bypassing Cursor’s proxy - No Cursor credit consumption for custom key usage

Auto Mode

The “Auto” model is Cursor’s cost-optimized routing system: - Approximately **\$0.25/M tokens** (cache read), **\$1.25/M tokens** (input), **\$6.00/M tokens** (output) - Unlimited on all paid plans - Routes to the most cost-effective model for the task at hand - Recommended for most daily use to conserve premium credits

6. Rules System

Overview

Cursor Rules are instructions injected into the LLM system prompt to control AI behavior. They are the Cursor equivalent of Claude Code's `CLAUDE.md` files.

Rule Types

Type	Trigger	Requires Description	Requires Glob
Always	Automatically included in every context	No	No
Auto Attached	Included when referenced files match a glob pattern	No	Yes
Agent Requested	AI decides whether to include (based on description)	Yes	No
Manual	Only used when explicitly invoked with <code>@ruleName</code>	Yes	No

Priority order: Manual (local) > Auto Attached > Agent Requested > Always

File Locations

Project Rules (recommended): - Stored in `.cursor/rules/` directory - Each rule is a `.mdc` file (MDC = Metadata + Content format) - Version-controlled, scoped to the project - Can reference other files (e.g., `@service-template.ts`) as additional context

User/Global Rules: - Defined in **Cursor Settings > Rules** - Apply across all projects for that user

Team Rules: - Defined in the Cursor cloud dashboard by admins - Automatically distributed to all team members - Can be marked as “recommended” or “required”

Legacy `.cursorsrules`: - Single file in project root (deprecated) - Still supported but migration to `.cursor/rules/` is recommended

MDC Format

```
---
description: Rules for React component files
globs: ["src/components/**/*.tsx"]
alwaysApply: false
---

# Component Guidelines

- Use functional components with TypeScript
- Always export with named exports
- Use @component-template.tsx as the base pattern
```

Generating Rules

Rules can be generated directly in conversations using the `/Generate Cursor Rules` command, useful when you have made decisions about agent behavior and want to reuse them.

7. MCP Support

Overview

Cursor supports the **Model Context Protocol (MCP)** for connecting to external tools and data sources. MCP acts as a plugin system for Cursor's Agent, enabling it to interact with databases, APIs, and third-party services.

Supported Transport Types

- **stdio**: Local servers that run on your machine, streaming via standard input/output
- **Streamable HTTP**: Independent processes that can handle multiple client connections

Configuration

MCP servers are configured in project or user settings files. Configuration specifies: - Server name and transport type - Command to launch the server (for stdio) - URL endpoint (for HTTP) - Authentication credentials - Environment variables

Capabilities and Limitations

- Cursor supports up to **40 tools** from MCP servers
- Currently **only tools are supported** (no resources yet – planned for future)
- MCP tools can also be configured and managed from the Cursor CLI

Available Integrations

A rich ecosystem of MCP servers exists for Cursor: - **Developer tools**: GitHub, GitLab, Linear, Jira - **Design**: Figma - **Databases**: PostgreSQL, Neon, Tinybird - **Cloud**: Google Drive, AWS, Stripe - **Browsers**: Browser-use for web interaction - **Custom**: Any MCP-compatible server

Comparison with Claude Code MCP

Both Cursor and Claude Code support MCP. Cursor's implementation is GUI-configured and limited to 40 tools. Claude Code configures MCP in `.mcp.json` files and has no documented tool limit.

8. Agent Mode Deep Dive

Architecture

Agent mode is Cursor's most autonomous feature. It operates as a **plan-execute-verify loop**:

```
User Request
|
v
[Plan] -- Decompose into sub-tasks
|
v
[Search] -- Find relevant files, docs, web resources
|
v
[Execute] -- Edit files, run commands, create files
|
v
[Verify] -- Check lints, run tests, review output
|
v
[Iterate] -- Fix any issues found
|
v
```

Present results to user with diffs

Tool Access

The Agent has access to the following tools:

Tool	Purpose
File Read	Read file contents
File Write/Edit	Modify or create files
File Delete	Remove files
File Search	Fuzzy-match file names
Codebase Search	Semantic search via embeddings
Terminal	Execute shell commands (sandboxed)
Web Search	Search the internet for information
Browser	Open and interact with web pages (for testing)
Linters	Check for code errors and warnings
MCP Tools	Any tools from configured MCP servers

YOLO Mode

When enabled, Agent **automatically runs terminal commands** without asking for approval. This includes: - Installing packages - Starting/stopping dev servers - Running tests - Executing build commands - Reading terminal output and iterating on errors

Checkpoints

Before making changes, Agent creates checkpoints so you can revert to any previous state. This is a safety net for autonomous operations.

Multi-Agent Parallel Execution (Cursor 2.0+)

The defining capability of Cursor 2.0: - Run up to **8 agents in parallel** on the same task - Each agent operates in an **isolated workspace** (git worktrees or remote VMs) - Compare and merge the best outcome - Enables “what if” exploration: run several repair strategies, refactor variants, or test pipelines simultaneously

Comparison with Claude Code Agent

Dimension	Cursor Agent	Claude Code
Interface	GUI (editor-embedded)	CLI (terminal-native)
Approval	Per-diff in editor (or YOLO)	Per-command in terminal (or -dangerously-skip-permissions)
Context Window	70K-120K effective (advertised 200K)	Consistent 200K
Multi-agent	Up to 8 parallel agents	Background agents via worktrees
Browser tool	Built-in native browser	Via MCP (e.g., Puppeteer)
File editing	Inline diffs with accept/reject	Applies edits directly
Token efficiency	Higher token usage	5.5x fewer tokens for identical tasks (per independent tests)
Hooks/Automation	Hooks (beta)	Hooks (stable)

9. Extensions & Marketplace

VS Code Extension Compatibility

As a VS Code fork, Cursor supports most VS Code extensions. However, there are important caveats:

Marketplace Access: - Microsoft's terms restrict the official VS Code Marketplace to "in-scope products" (VS Code, Visual Studio, GitHub Codespaces, Azure DevOps) - Cursor uses the **Open VSX Registry** instead of the Microsoft marketplace - Not every VS Code extension is available on Open VSX

Microsoft Extension Blocking (Late 2025): - Microsoft added activation checks to some first-party extensions (C/C++, Python debugger, etc.) - These extensions detect they are running in a fork and refuse to activate - This broke previously-working extensions for Cursor users

Workarounds: - Extensions can be manually compiled as VSIX packages and sideloaded - Many popular extensions have Open VSX equivalents - Community forks of blocked extensions exist

Cursor-Specific Features

Cursor does not have its own extension marketplace. Instead, it provides AI features natively that would otherwise require extensions: - AI autocomplete (replaces Copilot extension) - Inline chat (replaces extension-based chat) - Code actions (replaces various AI extensions)

Migration from VS Code

Cursor provides a one-click migration tool that imports: - All installed extensions - Settings and keybindings - Themes and configurations - The `code` CLI command works identically to `cursor`

10. Terminal Integration

AI-Powered Terminal

Cursor's terminal is deeply integrated with the AI system:

Terminal Cmd+K: Type a natural language description and Cursor generates the appropriate terminal command.

Terminal Output Reading: The AI can read terminal output natively, which enables: - Automatic error detection from compilation/test failures - Generating fixes based on exact error messages - Re-running commands after applying fixes

YOLO Mode: When enabled, the AI automatically executes terminal commands without prompting for approval. It will: - Install packages - Start/stop dev servers - Run builds and tests - Read output and iterate on failures

Sandboxed Terminals

Introduced in Cursor 1.7, sandboxed terminals add safety: - Commands run in an isolated environment - Exit codes are captured for AI reasoning - Background execution support - Error-aware command suggestions

Terminal in Agent Mode

When Agent mode uses the terminal: - Commands are displayed to the user before execution (unless YOLO) - Output is captured and fed back into the agent's context - The agent can chain multiple commands based on results - History is preserved within the agent session

11. Multi-File Editing

How Composer Handles Multi-File Edits

Composer (now integrated into Agent) is Cursor's approach to coordinated multi-file editing:

1. **Instruction Phase:** User describes the desired change (e.g., "Add authentication to the API routes and create a middleware file")
2. **Planning Phase:** The AI analyzes the codebase and identifies all files that need changes
3. **Execution Phase:** Changes are generated for all files simultaneously
4. **Review Phase:** All diffs are presented in a unified view

5. **Apply Phase:** User can accept/reject per file or per hunk

Diff Rendering

- Changes appear as **inline overlays** in the editor (green for additions, red for deletions)
- No separate diff window needed
- Accept with Tab, reject with Esc
- Can review changes file-by-file or accept all at once

Fast Apply

The specialized Fast Apply model ensures that multi-file edits are applied quickly: - Full file rewrites for files under ~400 lines - Diff-based application for larger files - ~1,000 tokens/second throughput - Speculative edits algorithm for speed optimization

Atomic Operations

Multi-file edits are treated as atomic operations: - All changes are part of a single checkpoint - Reverting rolls back all files to their pre-edit state - The AI considers cross-file dependencies when generating changes

12. Session Management

Current State

Cursor's session management has notable gaps compared to competitors:

Within a session: - Full context retention across the conversation - AI remembers earlier decisions, code changes, and discussion points - Checkpoint history for rollback

Between sessions: - Cursor AI **does not persist memory across conversations** - Starting a new chat loses all previous context - No equivalent to Claude Code's auto-memory system - Chat history is scoped to workspaces – changing workspace organization can lose chats

Known Issues

- Long-term projects suffer from re-explanation overhead after each restart
- Users report losing chat history containing critical architectural decisions
- Chat history persistence bugs have been reported even with settings enabled

Workarounds

- **Export Chat:** Save conversations as Markdown files via the “...” menu
- **Cursor Rules:** Encode persistent decisions as rules in `.cursor/rules/`
- **MCP-based memory:** Third-party MCP servers can provide cross-session memory
- **Notepad feature:** Cursor has a Notepad for persisting context, but it is manual

Comparison with Claude Code

Claude Code has a robust memory system: - `CLAUDE.md` files at project, user, and global levels - Auto-memory that persists learnings across sessions - Session resume with `--resume` flag - Memory files that accumulate project knowledge over time

Cursor has no equivalent automatic memory system. This is one of its most significant gaps.

13. Collaboration & Team Features

Team Rules

- Defined in the Cursor **cloud dashboard** by team admins
- Automatically distributed to all team members
- No local file storage needed for team-wide policies
- Rules can be marked as “recommended” or “required”
- Team rules also apply to Bugbot for consistent PR review behavior

Custom Commands

- Stored in `.cursor/commands/*.md` files
- Triggered with `/commandName` in the Agent input
- Reusable prompt templates shareable across the team
- Introduced in Cursor 1.6

Admin Controls

Feature	Business	Enterprise
Centralized billing	Yes	Yes
Team rules	Yes	Yes
Usage analytics dashboard	Basic	Advanced
Extension allowlisting	No	Yes
Model access control	No	Yes
MCP controls	No	Yes
SAML SSO	No	Yes
SCIM provisioning	No	Yes
Compliance policies	No	Yes
Secret scrubbing	No	Yes
Audit logging	No	Yes

Enterprise Dashboard

Enterprise admins get analytics including: - Lines of code accepted from AI suggestions - Lines deleted from AI suggestions - Most-used models per team - Usage patterns and credit consumption - Agent task completion rates

14. Background Agents

Overview

Background agents are **asynchronous remote agents** that run in isolated cloud environments, independent of your local editor. They are one of Cursor’s most powerful features for parallelizing development work.

How They Work

1. **Trigger:** Hit `Ctrl+E` or use the background agent tab in the sidebar
2. **Clone:** The agent clones your repo from GitHub into a remote VM
3. **Execute:** Works on a separate branch, making changes autonomously
4. **Push:** Pushes completed changes to your repo for review
5. **Handoff:** You review the diff and merge when satisfied

Environment

- Runs in an **isolated Ubuntu-based machine** by default
- Has **internet access** and can install packages
- Auto-runs all terminal commands (no approval needed, unlike foreground agent)

- Configurable via `Dockerfile` and `setup.sh` for custom environments

Configuration Requirements

- **Privacy Mode must be off** (code is sent to remote environments)
- GitHub repo must grant **read-write privileges**
- Environment can be customized with `environment.json`

Parallel Agents (Cursor 2.0+)

- Up to **8 agents in parallel** on a single problem
- Each agent operates in its own isolated workspace
- Can use git worktrees or dedicated remote VMs
- Cursor can automatically select the best result from parallel runs
- Enables “small team of agents” workflow rather than single-thread chat

Linear Integration (Cursor 1.5)

Background agents can be launched directly from **Linear issue tickets**, connecting project management to automated code generation.

Comparison with Claude Code Background Agents

Dimension	Cursor Background Agents	Claude Code Background Agents
Environment	Remote Ubuntu VMs	Local git worktrees
Internet access	Yes	Yes
Auto-execution	All commands auto-run	Configurable
Parallelism	Up to 8 agents	Multiple worktrees
Git integration	Clones repo, pushes to branch	Works in local worktree
Privacy	Code sent to remote environment	Code stays local
Issue tracker	Linear integration	GitHub via MCP

15. Bugbot (PR Review)

Overview

Bugbot is Cursor’s automated PR review system that integrates directly with GitHub and GitLab.

How It Works

1. A PR is opened or updated
2. Bugbot **automatically reviews** the diff
3. It analyzes the changes in context of the broader codebase
4. Leaves **inline comments** at the exact location of each issue
5. Identifies: logic bugs, edge cases, security issues, code quality problems
6. **Autofix**: Spins up isolated cloud agents in their own VMs that can fix issues and push commits to the PR branch

Configuration

- Supports **GitHub.com**, **GitLab.com**, **GitHub Enterprise Server (v3.8+)**, **self-hosted GitLab**
- Can run automatically on every PR update, or on-demand via comment (`cursor review` or `bugbot run`)
- Team rules apply to Bugbot for consistent behavior across repos

Effectiveness

- **70%+ of flagged issues** get resolved before merge
 - Finds issues not just in touched files but in how changes interact with existing code
 - Catches cross-component dependency issues
-

16. Cursor CLI

Overview

Cursor CLI is a command-line interface that brings Cursor’s AI agent capabilities to the terminal. It was introduced as a complement to the GUI editor.

Commands

```
# Open files and folders
cursor file.js           # Open a single file
cursor ./my-project      # Open a folder
cursor .                 # Open current directory

# Agent mode in terminal
agent chat "find one bug and fix it"

# Headless mode for CI/CD
cursor --headless --prompt "Run tests and fix failures"
```

Capabilities

- **Interactive mode:** Real-time conversation with the AI agent, with context retention across prompts
- **Non-interactive mode:** Single-shot commands for scripting and automation
- **Headless mode:** For CI/CD pipelines, no GUI required
- File reading, modification, deletion
- Shell command execution (with approval or auto-run)
- MCP tool configuration from the CLI
- Project opening and management

Comparison with Claude Code CLI

Cursor CLI is a newer addition, while Claude Code was **built as a CLI from day one**. Key differences:

Dimension	Cursor CLI	Claude Code
Maturity	Newer, evolving	CLI-first, mature
Primary mode	GUI editor with CLI complement	CLI-native with SDK
Headless support	Yes (newer feature)	Yes (core design)
SDK/API	No public SDK	Full SDK for programmatic use
CI/CD integration	Basic	Deep (GitHub Actions, hooks)
Session resume	No	Yes (--resume)

17. Pricing

Tier Comparison (as of early 2026)

Plan	Price	Key Inclusions
Hobby	Free	Limited Agent requests, limited Tab completions, no credit card required
Pro	\$20/month	Unlimited Tab, unlimited Auto mode, \$20 monthly credit pool for premium models
Pro+	\$60/month	Everything in Pro, \$60 monthly credit pool (3x Pro)
Ultra	\$200/month	Everything in Pro, 20x Pro usage, priority access to new features
Business	\$40/seat/month	Pro-equivalent AI, admin controls, centralized billing, team rules
Enterprise	Custom pricing	SSO/SAML, SCIM, compliance, pooled usage, audit logs, custom contracts

Annual billing saves **20%** across all paid tiers.

Credit System (Post-June 2025)

Cursor overhauled pricing from fixed “fast request” allotments to **usage-based credit pools**:

- Each paid plan includes a monthly credit pool equal to the plan price
- **Auto mode is unlimited** on all paid plans
- Manually selecting premium models draws from your credit balance
- Credits deplete based on actual token consumption at model-specific rates

Approximate Requests per \$20 Credit Pool

Model	~Requests per \$20
Gemini 2.0 Flash	~550
GPT-4o-mini	~400+
Claude Sonnet	~225
GPT-4o	~200
Claude Opus	~80-100

Auto Mode Pricing

- Cache read: ~\$0.25/M tokens
- Input: ~\$1.25/M tokens
- Output: ~\$6.00/M tokens

18. Strengths

1. Visual, Low-Friction Editing

The inline diff rendering, Tab autocomplete, and GUI-first design make Cursor extremely approachable. Developers see AI suggestions overlaid directly in their code and accept/reject with single keystrokes.

2. Multi-File Composer

Composer’s ability to generate coordinated edits across multiple files, with a unified review interface, is a genuine killer feature for feature implementation and refactoring.

3. VS Code Familiarity

By forking VS Code, Cursor inherits the world's most popular editor. Migration is a one-click import. Most extensions work. Muscle memory carries over.

4. Background Agents

The ability to spin up isolated remote agents that work asynchronously on tasks is powerful for parallelizing development work. Up to 8 parallel agents is unique in the space.

5. Tab Autocomplete Quality

The proprietary Tab model that predicts edits (not just insertions) and anticipates cursor movement is genuinely superior to basic completion engines.

6. Codebase Context

The embedding-based codebase indexing with Merkle tree efficiency, Turbopuffer vector search, and privacy-preserving obfuscation is well-engineered.

7. Built-in Browser Tool

Agent mode's native browser for testing web applications is useful for frontend development workflows.

8. Bugbot

Automated PR review with autofix capability is a differentiator for team workflows.

9. Enterprise Features

SSO, SCIM, compliance policies, usage analytics, team rules, and admin controls make Cursor viable for large organizations.

10. Rapid Innovation

The release cadence is extraordinary – major features ship every few weeks, from background agents to parallel agents to hooks to custom slash commands.

19. Limitations

1. Effective Context Window

Users consistently report hitting context limits at **70K-120K tokens** despite advertising 200K. Internal truncation and performance safeguards silently reduce effective context. This is a critical issue for large codebases.

2. Extension Ecosystem Friction

Microsoft's blocking of first-party extensions (C/C++, Python debugger) from running in forks is an ongoing and worsening problem. The Open VSX registry has gaps compared to the official VS Code marketplace.

3. No Persistent Memory

Cursor has no automatic memory system across sessions. Every new conversation starts from scratch, requiring re-explanation of project context, conventions, and decisions. Rules partially mitigate this but require manual maintenance.

4. Performance on Large Codebases

Developers report lag and freezing when working with large, complex codebases. The IDE becomes sluggish as conversation rounds grow during AI interactions.

5. Privacy Concerns

AI features require sending code to external servers. Background agents require disabling Privacy Mode entirely. For enterprises handling sensitive code, this is a fundamental tension.

6. Closed Source

Cursor is closed-source. Users cannot inspect the AI integration code, audit data handling, or contribute to the core product. This contrasts with some competitors offering open-source clients.

7. Upstream Merge Burden

VS Code releases monthly. Anysphere must constantly merge upstream changes, which is resource-intensive and occasionally introduces regressions. A dedicated team exists just for this maintenance.

8. AI Reliability

Common complaints include: - Misunderstanding context and inventing non-existent APIs - Producing subtle bugs that require careful review - Inconsistent quality across different programming languages - Better for Python/JS/TS than for Rust, Go, or less common languages

9. UI Clutter

Power users report too many popups, tabs, and “Fix with AI” buttons. Shortcut conflicts (Ctrl+K, Cmd+K) disrupt longtime VS Code habits.

10. No SDK/Programmatic API

Unlike Claude Code (which has a full SDK for programmatic use), Cursor has no public SDK for building integrations, plugins, or custom workflows programmatically.

11. Enterprise Maturity Gaps

Reports of unreliable AI support agents providing misinformation, slow human support for paying users, and compliance gaps for regulated industries.

20. Version History

Key Releases Timeline

Version	Date	Key Features
Initial Launch	March 2023	Public launch; VS Code fork with AI chat and autocomplete
0.44	December 2024	Agent Terminal: exit code capture, background execution (YOLO), editable commands, error awareness
0.45	January 2025	.cursor/rules directory for multiple rule files, Fusion Tab model, long context mode, MCP support, DeepSeek R1/v3 support
0.50	~Mid 2025	Background Agents launch (remote async agents)
1.5	~Mid 2025	Linear integration for background agents from issue tickets

Version	Date	Key Features
1.6	~September 2025	Custom slash commands (<code>.cursor/commands/*.md</code>), <code>/summarize</code> command for context limits
1.7	~October 2025	Agent Autocomplete, Hooks (beta), Team Rules, sandboxed terminals
2.0	October 29, 2025	Landmark release: Composer model (proprietary MoE), multi-agent parallel execution (up to 8), agent-centric interface redesign, native browser tool, sandboxed terminals GA
2.3	~Early 2026	Continued refinements, stability improvements
2.6	March 3, 2026	Interactive UIs in agent chats, team-shared private plugins, Debug mode
Composer 2	March 19, 2026	Frontier-level coding performance upgrade to the Composer model

Milestone Timeline

Date	Milestone
April 2022	Anysphere founded, \$400K pre-seed
2023	OpenAI accelerator graduation, public launch
January 2025	\$100M ARR (zero marketing spend)
June 2025	\$900M Series C, \$9.9B valuation
October 2025	Cursor 2.0 release
November 2025	\$2.3B Series D, \$29.3B valuation, \$1B+ ARR
Late 2025	1M+ daily active users, 300+ employees
March 2026	Composer 2 model, continued rapid iteration

21. Cursor vs Claude Code

Architectural Philosophy

Cursor: Embed AI into the visual editor experience. The developer stays in the IDE, selects code, prompts adjustments, and sees changes applied inline. Intelligence enhances the editing surface.

Claude Code: Operate at the system layer. The AI is a terminal-native agent that decomposes tasks, plans execution, edits files, and runs commands. It responds to objectives rather than reacting to typing.

Head-to-Head Comparison

Dimension	Cursor	Claude Code
Interface	GUI (VS Code fork)	CLI (terminal-native)
Primary strength	Interactive editing, visual diffs	Deep reasoning, multi-step execution
Context window	70-120K effective	Consistent 200K
Token efficiency	Higher usage	5.5x fewer tokens (independent tests)
Autocomplete	Best-in-class Tab model	None (not an editor)
Multi-file editing	Composer with visual diffs	Direct file edits with confirmation

Dimension	Cursor	Claude Code
Background agents	Remote VMs, up to 8 parallel	Local worktrees
Memory	No persistent memory	Auto-memory, CLAUDE.md, session resume
Rules	.cursor/rules (.mdc files)	CLAUDE.md (Markdown)
MCP support	Yes (40 tool limit)	Yes (no documented limit)
Hooks	Beta	Stable
SDK/API	No	Full SDK
CI/CD	Basic headless CLI	Deep integration
PR review	Bugbot (automated)	Manual or via skills
Browser tool	Native built-in	Via MCP
Extension ecosystem	VS Code extensions (with gaps)	MCP servers
Pricing (individual)	\$20/month Pro	\$20/month (Max plan with API key)
Pricing (team)	\$40/seat/month	\$30/seat/month (Team)
Open source	No	No (but open-source SDK)

Where Each Excels

Use Cursor for: - Day-to-day coding and rapid iteration - Interactive styling and frontend development - Quick file edits with visual confirmation - Developers who prefer GUI workflows - Teams wanting automated PR review (Bugbot) - Parallel exploration with background agents

Use Claude Code for: - Complex multi-step refactors and architecture tasks - Large codebase reasoning (200K consistent context) - CI/CD integration and automation - Headless/SDK-based workflows - Projects requiring persistent memory across sessions - Token-sensitive workloads - Terminal-first developers

The Emerging Consensus

Many development teams in 2026 use both: - **Cursor for everyone** on the team for daily development (\$40/seat) - **Claude Code for senior engineers** doing architecture, complex debugging, and code reviews - Combined cost of ~\$60-70/month per developer covers both use cases

This is not an either/or decision. They solve fundamentally different problems and complement each other well.

Sources

- [Cursor Official Site](#)
- [Cursor Documentation](#)
- [Cursor Changelog](#)
- [Cursor Blog - Shadow Workspace](#)
- [Cursor Blog - Cursor 2.0](#)
- [Cursor Blog - Series D](#)
- [Cursor Blog - CLI](#)
- [Cursor Blog - Secure Codebase Indexing](#)
- [Anysphere - Wikipedia](#)
- [Cursor \(code editor\) - Wikipedia](#)
- [Contrary Research - Cursor Business Breakdown & Founding Story](#)
- [How Cursor Serves Billions of AI Code Completions Every Day - ByteByteGo](#)
- [How Cursor Indexes Codebases Fast - Engineer's Codex](#)
- [How Cursor Actually Indexes Your Codebase - Towards Data Science](#)
- [How Cursor Built Fast Apply Using Speculative Decoding - Fireworks AI](#)
- [CNBC - Cursor Announces Major Update](#)
- [CNBC - AI Startup Cursor Raises \\$2.3B](#)
- [TechFundingNews - Anysphere MIT-Born Startup](#)
- [Cursor: How Forking VS Code Built a \\$29B Company - MMNTM](#)

- [Claude Code vs Cursor: The Real Difference - Emergent](#)
- [Claude Code vs Cursor: Complete Comparison - Northflank](#)
- [Claude Code vs Cursor: What to Choose - Builder.io](#)
- [Cursor vs VS Code: AI Coding Editor Showdown - Augment Code](#)
- [Cursor AI Review - Skywork](#)
- [Cursor Pricing Explained - Vantage](#)
- [VS Code Extension Marketplace Wars - DevClass](#)
- [Cursor Limitations Guide - P0stman](#)
- [Cursor Rolls Out Hooks, Team Rules - TestingCatalog](#)
- [What are Cursor Rules - WorkOS](#)
- [How to Write Great Cursor Rules - Trigger.dev](#)
- [Cursor Bugbot](#)
- [Using Cursor Bugbot to Autoreview Claude Code PRs - WorkOS](#)